

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. XML Parsing Algorithm

Algorithm: Parse and Display XML User/Product Details

Input: XML document containing structured user/product data

Output: Extracted data displayed in readable format

BEGIN

Step 1: Start the program.

Step 2: Specify the path of the XML document.

Step 3: Load the XML document into memory.

Step 4: Check whether the XML document is loaded successfully.

IF loading fails THEN

Display "Error: Unable to load XML file"

STOP

END IF

Step 5: Access the root element of the XML document.

Step 6: Traverse through the child nodes of the root element.

Step 7: FOR each child node DO

Identify the type of node.

IF the node contains user/product details THEN

Access its child elements.

Extract required values such as:

- ID
- Name
- Email
- Price
- Category

Store the extracted values.

Display the values in a readable format.

END IF

END FOR

Step 8: If a node contains nested elements,
recursively traverse the nested elements.

Step 9: Continue traversal until all required nodes
have been processed.

Step 10: Display all extracted information.

Step 11: Close or release the XML document resources.

END

Flow:

Load XML → Access Root → Traverse Nodes → Extract Data → Process Nested Nodes →
Display Output

2. MVC Workflow

Algorithm: MVC Request–Response Workflow

Input: User request

Output: Response generated by the View

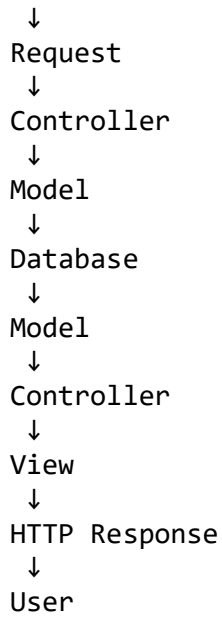
BEGIN

- Step 1: User sends a request from the browser.
- Step 2: Web server receives the HTTP request.
- Step 3: Identify the requested URL and HTTP method.
- Step 4: Forward the request to the appropriate Controller.
- Step 5: Controller receives the request.
- Step 6: Controller validates and interprets the request.
- Step 7: Controller calls the appropriate Model.
- Step 8: Model receives the required operation.
- Step 9: Model establishes communication with the database.
- Step 10: Execute the required database operation.
- Step 11: Retrieve the required data from the database.
- Step 12: Model processes the retrieved data.
- Step 13: Model returns the result to the Controller.
- Step 14: Controller passes the processed data to the View.
- Step 15: View uses the data to generate the required HTML/web page.
- Step 16: Web server sends the generated response to the browser.
- Step 17: Browser displays the response to the user.

END

MVC Flow:

User



3. JSP Request Flow

Algorithm: JSP Request–Response Processing

Input: Client request for a JSP page

Output: Dynamic HTML response

BEGIN

Step 1: User enters a JSP URL in the browser.

Step 2: Browser creates an HTTP request.

Step 3: Send the HTTP request to the web server.

Step 4: Web server receives the request.

Step 5: Identify the requested JSP page.

Step 6: Forward the request to the JSP/Servlet container.

Step 7: Check whether the JSP page has already been

converted into a Servlet.

```
IF JSP is not translated/compiled THEN
    Translate JSP into a Servlet.
    Compile the Servlet.
END IF
```

Step 8: Execute the generated Servlet.

Step 9: If required, invoke a Servlet or backend component.

Step 10: Backend component performs business processing.

Step 11: If database information is required,
establish database communication.

Step 12: Execute the required database query.

Step 13: Retrieve and process the database results.

Step 14: Pass the processed data to the JSP.

Step 15: JSP combines static HTML with dynamic data.

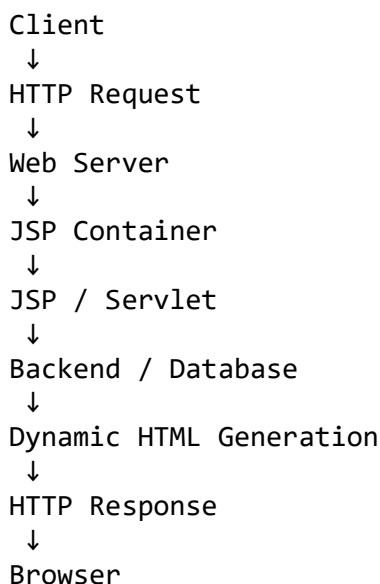
Step 16: Generate the final HTML response.

Step 17: Send the HTTP response to the browser.

Step 18: Browser receives and renders the dynamic web page.

END

JSP Flow:



4. Database Retrieval

Algorithm: Retrieve and Display Database Records

Input: SQL query

Output: Retrieved database records

BEGIN

Step 1: Start the database retrieval process.

Step 2: Load the required database driver.

Step 3: Specify the database URL, username,
and password.

Step 4: Establish a connection with the database.

Step 5: Check whether the connection is successful.

```
IF connection fails THEN
    Display "Database connection failed"
    STOP
END IF
```

Step 6: Create a statement or prepared statement.

Step 7: Write the required SQL SELECT query.

Step 8: Execute the SQL query.

Step 9: Store the returned records in a Result Set.

Step 10: Check whether the Result Set contains records.

```
IF no records are found THEN
    Display "No records found"
ELSE
    FOR each record in the Result Set DO
```

```
        Read the required fields.
```

```
        Extract values such as:
```

```
            ID
```

```
            Name
```

```
            Email
```

```
            Other required fields
```

```
        Display the retrieved values.
```

```
    END FOR
```

```
END IF
```

Step 11: Close the Result Set.

Step 12: Close the Statement.

Step 13: Close the database connection.

Step 14: End the database retrieval process.

END

Flow:

```
Start
↓
Load Driver
↓
Connect to Database
↓
Create Statement
↓
Execute SELECT Query
↓
Result Set
↓
Process Records
↓
Display Data
↓
Close Resources
↓
End
```

5. PHP Validation

Algorithm: Validate User Input Using PHP**Input:** User-submitted form data**Output:** Valid submission or appropriate error messages

BEGIN

Step 1: Start the form validation process.

Step 2: Receive the submitted form data.

Step 3: Initialize an empty error list.

Step 4: Read the Name, Email, and Password fields.

Step 5: Check whether the Name field is empty.

```
IF Name is empty THEN
    Add "Name is required" to error list.
END IF
```

Step 6: Check whether the Email field is empty.

```
IF Email is empty THEN
    Add "Email is required" to error list.
ELSE
    Check whether Email has a valid format.

    IF Email format is invalid THEN
        Add "Enter a valid email address"
        to error list.
    END IF
END IF
```


Step 7: Check whether the Password field is empty.

```
IF Password is empty THEN
    Add "Password is required" to error list.
ELSE
    Check password length.

    IF password length is less than the
        required minimum THEN
        Add "Password is too short"
        to error list.
    END IF
END IF
```

Step 8: Check whether the error list is empty.

```
IF error list is NOT empty THEN
    Display all appropriate error messages.
    Do not submit the form.
ELSE
    Accept the input.

    Sanitize/prepare the data.

    Process or store the valid data.

    Display "Form submitted successfully".
END IF
```

Step 9: End the validation process.

END

Validation Flow:

